

ORBIT

Team playbook: foundation reference

AI-powered global intelligence on a 3D globe. Global → Country → Event → Explanation → Ask ORBIT.

Team: Arham · Ayman · Affan · Shrey · Hardik

Generated 2026-09-19 from the Markdown sources in the repository. If this PDF and the repository disagree, the repository wins.

ORBIT team playbook

The foundation knowledge for the five ORBIT developers, in one place. Detailed rules live in the documents linked from each section. This playbook summarises them and never overrides them.

Document	What it is the authority on
AGENTS.md	Rules for people and coding agents: stack, folder ownership, contracts, styling, testing, Git
tasks.json	Who does what: every task, owner, branch, dependencies, acceptance criteria
docs/API.md	API routes, request/response shapes, data dictionary, mock data format
docs/INTEGRATION.md	Branches, commits, PRs, contract changes, merging
docs/DESIGN_SYSTEM.md	Tokens, primitives, UI states and conventions
docs/ALLOCATOR.md	The personal LLM prompt that tells each person what to do

1. Project overview

ORBIT is an AI-powered global intelligence platform. It combines geopolitical intelligence, healthcare intelligence, global news, geography, history/timelines and explainable AI around an interactive 3D globe.

Flow: Global → Country → Event → Explanation → Ask ORBIT.

Look: deep black, electric cyan, amber/orange. Futuristic, premium, cinematic and clean.

Constraints: 24-hour hackathon, five student developers, live demo on one laptop, must survive bad wifi. Our priorities, in order: reliability, speed, visual quality, easy collaboration, few merge conflicts, maintainability, demo quality.

2. Architecture



The reasons behind each part:

- **One small server:** it keeps API keys secret, avoids browser CORS blocks, and caches responses for the offline demo.

- **Seed data first:** the demo never depends on wifi. Live data only adds to it.
- **The mock AI** works with no key and is the automatic fallback.
- **No database:** the data is mostly read-only and small.

3. Stack (frozen)

Vite 8 · React 19 · TypeScript (strict) · react-globe.gl / three.js · Express 5 (tsx) · Zod 4 · TanStack Query 5 · Zustand 5 · React Router 7 · CSS Modules + tokens · Vercel AI SDK (ai , @ai-sdk/google) · Vitest 3 + Testing Library · ESLint 9 + Prettier · **npm** · **Node ≥ 22.12** (team uses 24). Details and forbidden alternatives: AGENTS.md section 2.

4. Repository map

AGENTS.md	CLAUDE.md	.cursor/rules/	Agent rules (AGENTS.md is the source)
README.md			Start here
tasks.json			Task manifest (who does what)
.env.example			Every environment variable, safe defaults
docs/			API, integration, design system, playbook, allocator, PDF
shared/			Contracts: schemas/ (one file per entity) + api.ts (routes,
envelope)			
server/			
index.ts	app.ts	env.ts	http.ts
routes/			
ask)			
data/store.ts	data/seed/*.json		
sources/			
ai/			
src/			
App.tsx	routes.ts	main.tsx	
api/client.ts			
layout/			
globe/			
features/<name>/			
ui/			
styles/			
state/			
test/			
public/data/countries.geojson			
scripts/			
mcp/	workshop-data/	.vscode/mcp.json	

	Server start, app, env parsing, response/error helpers
	countries, events, news, meta (health, sources), ai (insights,
	Seed loading + validation, curated data
	Live data + disk cache
	AI entry point, providers (mock, google)
	Routes (all declared once), URL helpers, bootstrap
	The only fetch
	App shell, global overview, 404
	The 3D globe
	country, event, timeline, news, health, insight, ask
	Shared UI primitives
	tokens.css, global.css, cssVar.ts
	Zustand store
	Test setup and helpers
	Country shapes (Natural Earth 110m, keyed by alpha-3 id)
	Tooling (PDF build, tasks.json test)
	Hackathon organiser tooling: do not touch

5. Contracts

The rules, summarised (full rules in AGENTS.md section 4 and docs/API.md):

- Every data type exists **once** as a Zod schema in shared/schemas/. The TypeScript type comes from z.infer.
- Import from @shared only. Never redefine a type. OrbitEvent, never Event.
- IDs: country = IND. Others = evt_, hs_, news_, src_, tl_, ins_ + lowercase slug.
- Dates are ISO UTC strings. Coordinates are {lat, lng}. Severity is an integer 1–5. Fields are camelCase. Enums are lowercase.
- Responses: { data } or { error: { code, message } }.
- Changing a contract → docs/INTEGRATION.md#contract-changes.

6. API contract and data dictionary

Route	Returns
GET /api/health	server status, data mode, AI provider
GET /api/countries , /api/countries/:id	Country list / one Country
GET /api/countries/:id/events , /api/countries/:id/timeline	a country's events / history
GET /api/events?kind= , /api/events/:id	all events (optionally by kind) / one event
GET /api/news?countryId=	headlines
GET /api/sources	sources
GET /api/insights/:subjectType/:subjectId	AI explanation for global/world , country/IND or event/<id>
POST /api/ask	AI answer to { question, context? }

Entity	Key fields
Country	id, name, region, capital, centroid, population, summary, riskLevel
OrbitEvent	kind (geopolitical health), id, title, summary, countryIds, location, occurredAt, severity, sourceIds, tags, plus kind-specific fields
NewsHeadline	id, title, url, sourceId, publishedAt, countryIds, eventId?
Source	id, name, url, type, reliability
TimelineEvent	id, countryId, date, datePrecision, title, description, eventId?
AllInsight	id, subjectType, subjectId, summary, keyPoints, sourceIds, confidence, provider, generatedAt

Full shapes, errors, relationships and the mock-data format: `docs/API.md` .

7. Environment setup

```
# once
git clone https://github.com/Ayman-Alam-India/orbit C:\dev\orbit # NOT inside OneDrive
cd C:\dev\orbit
node -v # must be 22.12 or newer
npm install
copy .env.example .env # optional: every value has a safe default

# every day
npm run dev # web on http://localhost:5173 + API on :8787
npm run check # lint + tests + build: must pass before any PR
```

Variable	Default	Meaning
PORT	8787	API server port (Vite proxies /api to it)
DATA_MODE	mock	mock = seed only, live = seed + live sources

Variable	Default	Meaning
AI_PROVIDER	mock	mock or google
GOOGLE_GENERATIVE_AI_API_KEY	empty	Only in your local .env , never committed
GOOGLE_MODEL	gemini-3.5-flash,gemini-3.5-flash-lite	Gemini model(s), tried in order
GROQ_API_KEY	empty	Groq key (free tier): the second model for claim verification
GROQ_MODEL	openai/gpt-oss-120b	Groq model used for verification

8. Design system

Tokens in `src/styles/tokens.css` : colour, severity scale, typography, spacing, radii, glows, motion, layers.
Primitives in `src/ui` : Panel, QueryState, Loader, ErrorState, ErrorBoundary, ItemList, SeverityBadge. Rule: CSS Modules + tokens only, no raw colours. Details: `docs/DESIGN_SYSTEM.md` .

9. Team ownership

Build model: Claude, running in Ayman's session, writes all the code. Each person below is the **decision owner** for their area: Claude asks them the customization questions (layout, content, wording, interactions, data choices) in plan mode before building, and they review and test the result. Teammates don't commit code. Full rule: `AGENTS.md` section 0.

Current roles (`tasks.json` → `team`):

Person	Role now
Hardik	Reviews and merges every PR, runs sync points and the demo laptop
Affan	Decides AI behaviour and tone, provides the AI key, reviews AI output
Arham	Decides visual direction, layout and globe look, reviews all UI
Ayman	Drives the Claude build session and relays decisions, decides the country, event and timeline views
Shrey	Decides which countries and events feature, verifies researched facts, runs the demo rehearsal

The table below maps each area to its decision owner (the "Owns" column is the code area they decide on and review).

Person	Primary responsibility	Owns	Depends on	Hands off to
Hardik (heavy)	Integrator: merges, contracts, API server, live data, demo hardening	<code>shared/</code> , <code>server/</code> <code>core</code> + <code>routes/</code> + <code>sources/</code> + <code>store.ts</code> , <code>src/App.tsx</code> , <code>src/routes.ts</code> , <code>src/api/</code> , <code>configs</code> , <code>package*.json</code> , <code>docs</code> , <code>tasks.json</code>	Everyone's PRs	Everyone (contract updates, merged main)
Affan (heavy)	AI: real provider, explanations, Ask ORBIT	<code>server/ai/</code> , <code>server/routes/ai.ts</code> , <code>src/features/insight/</code> , <code>src/features/ask/</code>	Hardik (contract changes), Arham (primitives, layout)	Ayman/Arham (InsightCard), Hardik (AI cache for demo)

Person	Primary responsibility	Owns	Depends on	Hands off to
Arham (medium-heavy)	Design system, app shell, cinematic globe, global view	<code>src/styles/</code> , <code>src/ui/</code> , <code>src/globe/</code> , <code>src/layout/</code> , <code>src/state/</code> , <code>public/</code>	Shrey (data density), Hardik	Everyone (primitives)
Ayman (medium)	Country, event and timeline views	<code>src/features/country/</code> , <code>src/features/event/</code> , <code>src/features/timeline/</code>	Arham (primitives), Affan (InsightCard), Shrey (data)	Arham (layout fit)
Shrey (light)	Curated data, news and health panels, demo script	<code>server/data/seed/</code> , <code>src/features/news/</code> , <code>src/features/health/</code> , <code>docs/DEMO.md</code>	Hardik (contracts)	Everyone (data), Hardik (demo script)

Shared files that need coordination: see `AGENTS.md` section 3 ("Shared hot files").

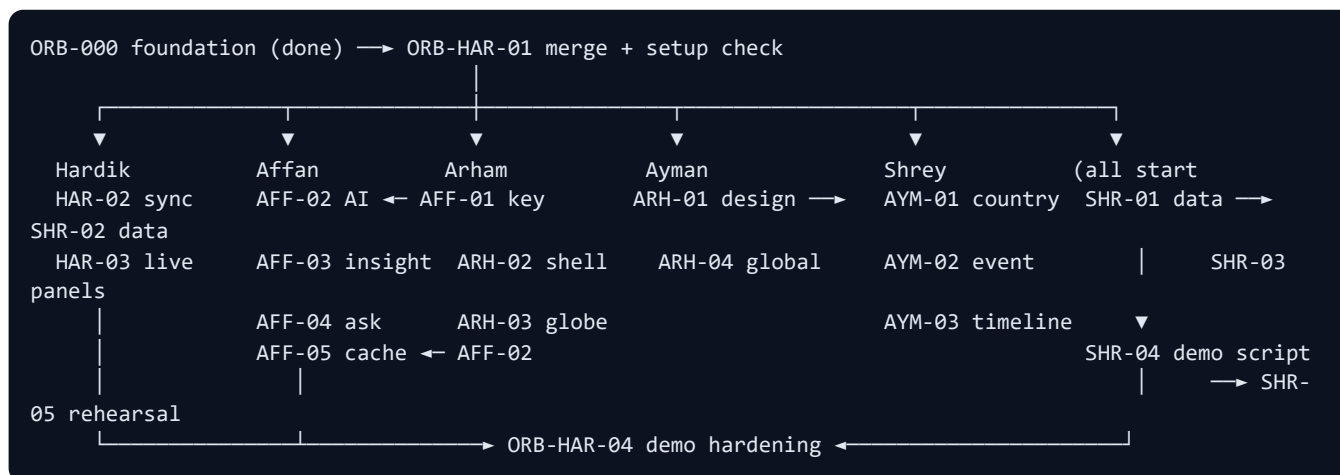
10. Task system

`tasks.json` is the only list of work. Each task has:

Field	Meaning
<code>id</code>	<code>ORB-<PERSON>-<NN></code> (e.g. <code>ORB-ARH-03</code>). <code>ORB-000</code> = foundation
<code>owner</code>	The one person responsible: decides and reviews if there is a <code>builder</code> , does the task if not
<code>builder</code>	Present when Claude builds the task (in Ayman's session)
<code>status</code>	<code>todo</code> , <code>in_progress</code> or <code>done</code> . Hardik updates it on merge (<code>tasks.json</code> is a shared hot file)
<code>priority</code>	<code>P0</code> demo breaks without it · <code>P1</code> should have · <code>P2</code> nice to have
<code>targetWindow</code>	Hours from the start (H0) when it should happen
<code>branch</code>	The Git branch to use
<code>paths</code>	The only files/folders the task may change
<code>dependsOn</code>	Tasks that must be merged first
<code>acceptance</code>	What "done" means
<code>tests</code>	Tests to add or keep green
<code>handoff</code>	Who to tell what when you finish

Work order (Claude for built tasks, each person for their own tasks): `P0` before `P1` before `P2`, and within a priority, the earliest `targetWindow` whose `dependsOn` tasks are done. `npm test` checks that `tasks.json` has valid IDs, owners and dependencies, with no cycles.

11. Dependency graph



Right after ORB-HAR-01, **all five people can work in parallel**. ORB-AFF-01 (getting the AI key) can start at H0.

12. Git workflow (summary)

Branch from `main` using the task's `branch` name, then make small commits (`area: change`). Before the PR, `git pull origin main` and run `npm run check` . The PR title is `[ORB-XXX-NN] summary` . Hardik squash-merges. Sync points happen every ~4h. Full rules: `docs/INTEGRATION.md` .

13. Integration workflow

1. Contract/API changes merge first (Hardik) → everyone pulls.
2. Server changes merge next, then UI.
3. After each merge batch, Hardik runs `npm run check` + the click-through on `main` .
4. A broken `main` is fixed or reverted before new work continues.

14. Testing

Layer	Tests (existing)	What they protect
Contracts	<code>shared/contracts.test.ts</code>	ID, date, severity and event-kind rules
Data	<code>server/data/seed.test.ts</code>	Every seed file is valid and every reference exists
API	<code>server/app.test.ts</code>	Every route returns contract-valid <code>{ data }</code> / <code>{ error }</code>
Frontend	<code>src/api/client.test.ts</code>	Envelope unwrapping, error codes
Integration	<code>src/App.test.tsx</code>	The real UI rendering real API data (globe mocked)
Manifest	<code>scripts/tasks.test.ts</code>	<code>tasks.json</code> IDs, owners and dependencies

Run all tests with `npm test` (or `npm run test:watch`). The gate is `npm run check` . Each task adds the tests listed in `tasks.json` .

15. Deployment plan

There is no cloud deployment. The demo runs with `npm run dev` on one laptop (ORB-HAR-04):

1. Pull the final `main`, `npm install`, `npm run check`.
2. Create the demo `.env` (`AI_PROVIDER=google` + key, `DATA_MODE=live` to warm caches).
3. Click through the full demo once online (fills `server/.cache/`), then **turn wifi off** and run it again.
4. Keep the backup recording (ORB-SHR-05) ready.

`npm run build` + `npm run preview` produces a static frontend build, but the API server is still needed, so we demo with `npm run dev`.

16. Coding-agent rules

`AGENTS.md` is read automatically by Codex and Cursor. `CLAUDE.md` imports it for Claude Code, and `.cursor/rules/orbit.mdc` points Cursor to it. The key rules:

- Work on one task from `tasks.json`, inside its `paths`.
- Import types from `@shared` only, and never invent tasks, fields, routes or packages.
- Use CSS Modules + tokens, and handle loading, error and empty states.
- Never commit secrets, and run `npm run check` before a PR.

To start an agent session with the right context, use `docs/ALLOCATOR.md`.

17. Common mistakes to avoid

Mistake	Instead
Writing <code>interface Country {...}</code> in a component	<code>import { type Country } from '@shared'</code>
Naming a type <code>Event</code>	<code>OrbitEvent</code>
<code>fetch('/api/countries')</code> inside a component	A hook in your feature folder using <code>apiGet(API_ROUTES.countries)</code>
Hard-coding <code>'/country/' + id</code>	<code>paths.country(id)</code> from <code>src/routes.ts</code>
<code>color: #00e5ff</code>	<code>color: var(--color-primary)</code>
Installing a package without a plan	State it in the plan, get Hardik's OK, install in the PR that needs it
Putting the AI key in code or a <code>VITE_</code> variable	Local <code>.env</code> only
A teammate committing code "to fix a small thing"	Post it in the chat or a PR comment; Claude makes the change
Country ID <code>in</code> , <code>India</code> or <code>ind</code>	<code>IND</code>
Dates like <code>19/09/2026</code>	<code>2026-09-19T00:00:00Z</code>
Deleting a failing test	Fix the code, or ask
Working inside OneDrive	<code>C:\dev\orbit</code>
A long-lived branch	PR every 2–4 hours

18. Definition of done

A task is done when:

- ☐ Every acceptance criterion in `tasks.json` is met
- ☐ The tests listed in the task exist and pass, and `npm run check` passes
- ☐ Only the task's `paths` changed
- ☐ The owner's customization decisions were asked in plan mode and are listed in the PR
- ☐ Loading, error and empty states are handled, using tokens and primitives only
- ☐ The PR is merged into `main` by Hardik, who sets `status` to `done` in `tasks.json`
- ☐ The handoff message is posted in the team chat

19. Personal LLM allocator

Each person pastes the prompt from `docs/ALLOCATOR.md` into their own LLM (Claude, ChatGPT, Cursor, Codex) with their name. It reads `tasks.json` and these docs, then tells them their role, tasks, branch, files, dependencies, acceptance criteria, tests and handoff. It never invents tasks.

ORBIT API and data contract

Owner: Hardik. The code is the truth: schemas live in `shared/schemas/`, paths in `shared/api.ts`, and routes in `server/routes/`. This document explains them in plain words. If you find a difference, the code wins, so tell Hardik and this file gets fixed.

How it fits together



- In development, the browser calls `http://localhost:5173/api/...` and Vite forwards it to `http://localhost:8787`.
- The browser never calls external APIs or AI directly. Keys stay on the server.

Response format (every route)

Success, HTTP 200:

```
{ "data": <the thing you asked for> }
```

Failure, HTTP 4xx/5xx:

```
{ "error": { "code": "NOT_FOUND", "message": "Country \"XXX\" not found" } }
```

code	HTTP	Meaning
VALIDATION	400	Bad input: wrong ID format, invalid body, invalid query
NOT_FOUND	404	The ID is valid but does not exist, or the route does not exist
UPSTREAM	502	A live source failed and there was no fallback (should be rare)
INTERNAL	500	A bug on the server (check the server terminal for <code>[api] unexpected error</code>)

In the frontend, `apiGet / apiPost` (`src/api/client.ts`) unwrap `data` for you and throw an `ApiClientError` (`code`, `status`, `message`) on failure. If the server is not running you get `code: "NETWORK"`.

Routes

Always build paths with `API_ROUTES` from `@shared`. The column "Builder" shows how.

Method + path	Builder	Returns (data)	Errors
GET /api/health	<code>API_ROUTES.health</code>	<code>HealthStatus { status: "ok", dataMode, aiProvider }</code>	none
GET /api/countries	<code>API_ROUTES.countries</code>	<code>Country[]</code>	none
GET /api/countries/:id	<code>API_ROUTES.country(id)</code>	<code>Country</code>	400 bad ID, 404
GET /api/countries/:id/events	<code>API_ROUTES.countryEvents(id)</code>	<code>OrbitEvent[]</code> newest first, every event whose <code>countryIds</code> includes <code>id</code>	400, 404
GET /api/countries/:id/timeline	<code>API_ROUTES.countryTimeline(id)</code>	<code>TimelineEvent[]</code> oldest first	400, 404
GET /api/events?kind=	<code>API_ROUTES.events(kind?)</code>	<code>OrbitEvent[]</code> newest first. <code>kind</code> optional: <code>geopolitical</code> or <code>health</code>	400 bad kind
GET /api/events/:id	<code>API_ROUTES.event(id)</code>	<code>OrbitEvent</code>	400 bad ID, 404
GET /api/news?countryId=	<code>API_ROUTES.news(countryId?)</code>	<code>NewsHeadline[]</code> newest first. Without <code>countryId</code> : all	400 bad ID
GET /api/sources	<code>API_ROUTES.sources</code>	<code>Source[]</code>	none
GET /api/insights/:subjectType/:subjectId	<code>API_ROUTES.insight(type, id)</code>	<code>AIInsight</code> . <code>subjectType</code> = <code>global</code> (id <code>world</code>), <code>country</code> (country ID) or <code>event</code> (event ID)	400 bad type, 404 unknown subject
POST /api/ask	<code>API_ROUTES.ask</code>	<code>AskAnswer</code>	400 invalid body
POST /api/speech	<code>API_ROUTES.speech</code>	<code>audio/wav</code> (not JSON): the narrator voice for <code>{ text }</code> (1–1500 chars). Gemini TTS, cached per sentence in <code>server/.cache/speech</code>	400 invalid body, 503 no key/quota (browser voice takes over)

Method + path	Builder	Returns (data)	Errors
GET /api/verify/:subjectType/:subjectId	API_ROUTES.verify(type, id)	VerificationReport : every claim of that insight checked by each verifier, with agreement	400 bad type, 404 unknown subject
GET /api/impacts?eventId=&countryId=	API_ROUTES.impacts(filter)	ImpactLink[] : ripple effects caused by an event, or touching a country (as cause, target or via its markets)	400 bad ID
GET /api/markets?countryId=	API_ROUTES.markets(countryId?)	MarketQuote[] : curated snapshot, live Yahoo Finance values in DATA_MODE=live	400 bad ID
GET /api/weather/:countryId	API_ROUTES.weather(id)	WeatherReport for the capital (Open-Meteo). 503 UPSTREAM when live data is off and nothing is cached	400, 404, 503
GET /api/scenarios	API_ROUTES.scenarios	Scenario[] : what-if chokepoint scenarios with sourced facts	none
GET /api/simulate? scenario=&brentPct=	API_ROUTES.simulate(id, pct)	SimulationResult : knock-on effects of a Brent shock (−30... +100%, clamped), each with formula and sources	400 bad input, 404 unknown scenario

POST /api/ask body (AskRequest):

```
{
  "question": "Why is dengue rising?",
  "context": { "countryId": "IND", "eventId": "hs_ind_dengue_surge" }
}
```

question is 1–500 characters. context and both of its fields are optional. With eventId, ORBIT answers about that event. With only countryId, it answers about that country. With neither, it answers about the world.

Shared rules for all data

Rule	Example
Country ID = ISO 3166-1 alpha-3, uppercase	IND, USA, BRA (matches properties.id in public/data/countries.geojson ; Kosovo is KOS)
Other IDs = prefix + lowercase slug (a-z, 0-9, _)	evt_usa_tariff_review, hs_ind_dengue_surge, news_..., src_..., tl_..., ins_...
Dates = ISO 8601 UTC with Z	2026-09-19T10:00:00Z
Coordinates = { lat, lng }	{ "lat": 28.61, "lng": 77.21 }

Rule	Example
Severity = integer 1–5	1 Low, 2 Guarded, 3 Elevated, 4 High, 5 Critical
Field names = camelCase. Enum values = lowercase	occurredAt , "geopolitical"

Entities (data dictionary)

Country (shared/schemas/country.ts)

Field	Type	Notes
id	CountryId	IND
name	string	Display name
region	string	e.g. South Asia
capital	string	
centroid	{lat,lng}	Where the globe camera flies to
population	positive integer	
summary	string	One or two sentences
riskLevel	1–5	Overall current risk

OrbitEvent (shared/schemas/event.ts)

Either a **GeopoliticalEvent** (kind: "geopolitical" , ID evt_...) or a **HealthSignal** (kind: "health" , ID hs_...). Check kind before using kind-specific fields.

Field	Type	Notes
kind	geopolitical health	Decides which extra fields exist
id	evt_... or hs_...	Prefix must match kind
title , summary	string	
countryIds	CountryId[] (≥ 1)	The first one is the primary country (used for links)
location	{lat,lng}	Marker position
occurredAt	ISO date-time	
severity	1–5	
sourceIds	SourceId[]	Must exist in sources.json
tags	string[]	Free-form
geopolitical only: category	string	Free text until the category list is agreed
geopolitical only: actors	string[]	Governments, organisations, people
health only: indicator	string	e.g. Dengue cases
health only: metric (optional)	{ value: number, unit: string }	e.g. { 18400, "cases this month" }

NewsHeadline (shared/schemas/news.ts)

`id` (`news_...`), `title`, `url`, `sourceId`, `publishedAt`, `countryIds` (may be empty), `eventId` (optional, links to an OrbitEvent), `publisher` (optional: the outlet's domain, e.g. `reuters.com`, set for live GDELT headlines whose `sourceId` is `src_gdelt`).

Source (shared/schemas/source.ts)

`id` (`src_...`), `name`, `url`, `type` (`news` | `government` | `international_org` | `research` | `dataset`), `reliability` (`high` | `medium` | `low`).

TimelineEvent (shared/schemas/timeline.ts)

`id` (`tl_...`), `countryId`, `date`, `datePrecision` (`day` | `month` | `year`), `title`, `description`, `eventId` (optional). A year-only fact is stored as 1 January of that year with `datePrecision`: "year".

AllInsight (shared/schemas/insight.ts)

`id` (`ins_...`), `subjectType` (`global` | `country` | `event`), `subjectId` (`world`, a country ID or an event ID), `summary`, `keyPoints` (≥ 1), `sourceIds`, `confidence` (`low` | `medium` | `high`), `provider` (`mock`, `google`, ...), `generatedAt`. It is explainable because it always says which sources it used, how sure it is, and which AI produced it.

AskRequest / AskAnswer (shared/schemas/ask.ts)

Request: `question`, optional `context` { `countryId?`, `eventId?` }. Answer: `answer`, `sourceIds`, `provider`, `generatedAt`.

VerificationReport (shared/schemas/verification.ts)

The insight for a subject is split into claims (summary sentences + key points, max 6). Each claim is checked **independently** by every verifier against ORBIT's data: `gemini` and `groq` (LLMs, when their key is set) and `rules` (deterministic: every figure and most key terms must appear in the data; always runs, works offline).

Fields: `subjectType`, `subjectId`, `insightProvider`, `models` (verifiers that answered), `unavailable` ({ `model`, `reason` } for missing keys or failures), `claims[]` (`id` `clm_N`, `text`, `verdicts[]` of { `model`, `verdict`, `sourceIds`, `reason` }, `agreement`: `agree` / `disagree` / `single`), `agreementScore` (share of multi-verifier claims where all agree, 0–1), `generatedAt`. Verdicts: `supported`, `unsupported` (not in the data), `contradicted`. A complete report is cached; if a model fails later (e.g. no wifi), the cached report is served.

ImpactLink: ripple effects (shared/schemas/impact.ts)

ORBIT's USP: how an event spreads to other countries and markets. `id` (`imp_...`), `eventId` (the cause), `target` ({ `kind`: 'country', `id` } or { `kind`: 'market', `id` }), `channel` (energy, shipping, trade, finance, health, security), `direction` (up, down, risk), `strength` (1–3), `effect` (a few words), `mechanism` (how), `basis`, `sourceIds`, optional `marketId` (a live figure that shows it) and `followsImpactId` (chains, e.g. Brent → India's crude bill → rupee). `basis`: `sourced` means a cited, opened source states the link. `basis`: `analysis` means ORBIT's reasoning from sourced facts. The UI and the AI always say which it is. The globe draws one arc per event → affected country.

MarketQuote (`shared/schemas/market.ts`)

`id` (`mkt_...`), `name`, `kind` (index, stock, commodity, fx, fuel), `symbol` (Yahoo ticker, if live-updatable), `countryId`, `value`, `currency`, `unit`, `previousClose`, `history` (1 month of daily closes), `asOf`, `sourceId`, `live`. Tracked: Brent, USD/INR, NIFTY 50, Sensex, Reliance, TCS, HDFC Bank, Infosys (Yahoo Finance) and Delhi petrol/diesel (curated from Business Today / Goodreturns, updated by hand). Live quotes refresh in the background every 15 min in `DATA_MODE=live`, and the curated snapshot is the offline fallback.

WeatherReport (`shared/schemas/weather.ts`)

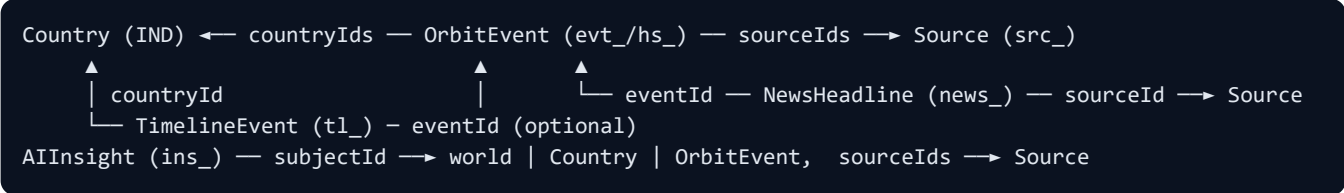
`countryId`, `place` (capital), `location`, `current` (temperature, precipitation, wind, WMO code + description), `daily` (7 days of max/min/precipitation), `asOf`, `sourceId` (`src_open_meteo`). Live in `DATA_MODE=live`, cached for 30 min and offline.

Scenario / SimulationResult: what-if simulator (`shared/schemas/simulation.ts`)

A **Scenario** (`scn_...`) is a chokepoint closure: `title`, `description`, `chokepoint` (name + location), `defaultBrentPct`, sourced `facts`, optional `flow` (million b/d through the chokepoint, pipeline bypass, share of world consumption), and `affected` countries (`exporter` / `importer` / `transit`, with a note and sources). Curated: Strait of Hormuz (EIA) and Bab el-Mandeb.

A **SimulationResult** is transparent arithmetic on the user's chosen Brent shock and live Brent and USD/INR: new Brent price, oil stranded after bypass, India's crude bill change (price only), and the upper-bound crude cost per litre ($\Delta\text{Brent} \times \text{USD/INR} \div 159 \text{ l/bbl}$). Each impact has a `formula`, `sourceIds` and `basis` (`simulation` or `analysis`). **It is a simulation, never a forecast**, and the UI and the AI both say so. `POST /api/ask` accepts `context.simulation { scenarioId, brentPct }` so Ask ORBIT can explain the simulation on screen.

Relationships



The server checks every one of these links when it starts, and `npm test` checks them too (`server/data/seed.test.ts`). A broken link stops the server with a message such as `news.json news_x: unknown source "src_missing"`.

Data files (`server/data/seed/` and `server/data/fixtures/`)

One JSON file per entity, each a plain **array** of objects matching the schema above:

File	Contains	Owner
<code>countries.json</code>	<code>Country[]</code>	Shrey
<code>events.json</code>	<code>OrbitEvent[]</code> (both kinds mixed, each with <code>kind</code>)	Shrey
<code>news.json</code>	<code>NewsHeadline[]</code>	Shrey
<code>sources.json</code>	<code>Source[]</code>	Shrey

File	Contains	Owner
<code>timeline.json</code>	<code>TimelineEvent[]</code>	Shrey

Curated seed (what the app shows): 15 countries, 14 real events (7 geopolitical, 7 health signals), 7 headlines and 11 timeline entries, current as of 19 September 2026. Sourcing rules:

- Every fact comes from a page that was actually opened. Each `sources.json` entry points to that exact page (WHO Disease Outbreak News, the UKHSA NaTHNaC outbreak list, the Federal Reserve, the UN, and named news articles).
- Populations (2024) and capitals come from the World Bank (`SP.POP.TOTL` and its country API), and `centroid` is the capital's location. Kosovo uses the map's `KOS` code (the World Bank uses `XXK`).
- `riskLevel` and event `severity` are **ORBIT's editorial ratings** based on the tracked events, not an official index.
- Headlines use the article's real title and URL. Dates are publication dates (time set to 00:00 UTC).

Test fixtures (`server/data/fixtures/`): a small, stable, clearly fake data set (India, US, Brazil with `example.org` sources). Tests always run against it (`ORBIT_SEED_DIR` in `vitest.config.ts`), so curating the real seed never breaks a test.

To check your edits, run `npm test` . The seed test validates both data sets, checks that every country exists on the globe map, and rejects placeholder (`example.org`) sources in the curated seed. It prints the exact file, item and field that is wrong.

Data modes and future live data

Setting	mock (default)	live
<code>DATA_MODE</code>	Seed data only, fully offline	Seed data + live sources (<code>server/sources/</code>)
<code>AI_PROVIDER</code>	Offline analysis built from seed data	<code>google</code> (Gemini). Falls back to mock on any failure

Rules for every live source (server-side only):

1. Fetch → validate with the shared schema (drop invalid items) → save with `writeCache(key, data)` .
2. Merge with seed data: seed is always the base, live adds to it.
3. On any failure: log `[sources]` ... and return `readCache(key)` , then seed. Never throw to the route.
4. Live items must follow the same ID rules (e.g. `news_<source>_<slug>`) and reference existing sources.
5. A new external API key = a new empty line in `.env.example` + a field in `server/env.ts` .

Live news (implemented): GDELT (`server/sources/gdelt.ts`), free and keyless.

- Feeds: one per country (query = the country's name) plus a global feed (the countries with severity 4–5 events). English-language articles from the last 7 days, max 10 per feed, `id = news_gdelt_<hash of URL>` .
- `GET /api/news` answers **immediately** with seed + cached live headlines. A stale or missing cache (over 30 min old) starts a background refresh, so GDELT's slowness never blocks the UI. Live headlines therefore appear on the next load.
- In `DATA_MODE=live` the server pre-fetches every feed on startup (`[sources]` warming live news...), one request every 10 s with a 20 s back-off on HTTP 429, because GDELT rate-limits hard. Start it well before the demo so the cache is full.
- Cached feeds live in `server/.cache/news-<COUNTRY|global>.json` . With wifi off, cached and seed headlines keep working.

Changing this contract

Adding a field, route or entity is a contract change. Follow [docs/INTEGRATION.md#contract-changes](#) . Hardik updates [shared/](#) , this document and the tests in one PR, which merges before the feature that needs it.

ORBIT integration contract

Owner: Hardik. This document describes how work becomes one working ORBIT app without breaking it.

Build model: Claude (in Ayman's session) is the only code writer (`AGENTS.md` section 0). Before each task, Claude asks the task owner the customization questions in plan mode. Teammates review PRs, test, and propose changes as PR comments or in the chat, never as commits. The steps below are what Claude follows. Hardik still reviews and merges everything.

The flow

```
main → your branch (from tasks.json) → commits → pull main + npm run check → PR → Hardik
merges (squash) → main
```

1. `git checkout main && git pull` then `git checkout -b <branch from tasks.json>` (e.g. `claude/country-view`).
2. Ask the task owner the customization questions (plan mode), then work only inside the task's `paths` (see `tasks.json` and `AGENTS.md` section 3).
3. Commit small and often.
4. Before the PR: `git pull origin main` (merge main into your branch), resolve conflicts, `npm run check`.
5. Open a PR into `main`. Hardik reviews, merges (squash), and posts in the chat.
6. Everyone: `git checkout main && git pull && npm install`, then start the next task on a new branch.

Branch naming

Exactly as in the task's `branch` field: `claude/<short-topic>` for Claude-built tasks. Examples: `claude/globe`, `claude/google-provider`. Use one branch per task and delete it after merge. Never commit directly to `main`.

Commits

- Format: `<area>: <what changed>` in the present tense, e.g. `country: show risk level`, `seed: add 10 countries`.
- One logical change per commit. Never commit generated or secret files (see "Never commit").

Pull requests

- Title: `[ORB-XXX-NN] short summary`, e.g. `[ORB-AYM-01] Country view`.
- The description says: the owner's customization decisions, what changed, how to test it (the click path), the tests you added, and any contract/API changes (these should already be merged separately), plus screenshots for UI.
- Keep PRs small: one task per PR. A PR that changes files outside its task's `paths` is sent back.
- Checklist before requesting a merge:
 - ☐ `npm run check` passes
 - ☐ Only my task's `paths` changed (plus tests)

- ☐ Loading / error / empty states handled
- ☐ Customization decisions came from the task owner and are listed
- ☐ No secrets, no `.env`, no new packages (unless stated in the plan and OK'd by Hardik)

Shared-file rules

Shared hot files (`AGENTS.md` section 3): `shared/**`, `package.json`, `package-lock.json`, `src/App.tsx`, `src/routes.ts`, `src/styles/tokens.css`, `src/ui/index.ts`, `server/app.ts`, `.env.example`, `tasks.json`, `AGENTS.md`.

- Change them in a **separate small PR** that is merged **before** the feature PR that needs it.
- Everything else **imports** from them freely.

Contract changes

A contract change is any change to `shared/` (a field, schema, enum, ID rule, route or response shape).

1. **Announce** in the team chat: `CONTRACT CHANGE: add optional imageUrl to Country, needed for ORB-AYM-01`.
2. **Claude** makes the change in one PR: `shared/` + `docs/API.md` + the server route or seed validation + tests. New fields start as `.optional()`, so existing data and code keep working.
3. Hardik merges it and posts `CONTRACT UPDATED: ... pull main`.
4. Everyone pulls main. Then the feature PR uses the new field.

Never change the meaning of an existing field and never delete a field while someone uses it. Add a new field instead.

API changes

Same process as contract changes. New routes are added to `API_ROUTES` (`shared/api.ts`) and the routes table in `docs/API.md` in the same PR as the server code. Frontend code uses a new route only after that PR is on `main`.

How conflicts are avoided

- One code writer (Claude in Ayman's session), so there are no parallel edits to the same file.
- One task per branch, and each task only touches its own `paths`.
- All routes are declared up front in `src/App.tsx`, so features never touch the router.
- Each feature has its own query hooks and keys. There is no shared query key file.
- Packages are only added in the PR that needs them, so there are no lockfile conflicts.
- CSS Modules per component, so there are no global CSS conflicts.

Teammates never commit, so if a teammate wants a change, they post it as a PR comment or in the chat.

Testing before merging

The author runs `npm run check` (lint, tests, typecheck, build) and clicks through their change in `npm run dev`. Hardik runs `npm run check` on `main` after each merge batch and clicks Global → Country → Event → Ask (task

ORB-HAR-02). If `main` breaks, fixing it comes before any new work. Revert the PR if it can't be fixed in 15 minutes.

Sync points

About every 4 hours (task ORB-HAR-02), Hardik merges ready PRs in order (`shared/` changes first, then server, then UI), and everyone pulls and tests on their own laptop. Branches stay short-lived.

Never commit

- `.env` or any real key or token (only `.env.example` with empty values)
- `node_modules/`, `dist/`, `server/.cache/`
- Another lockfile (`yarn.lock`, `pnpm-lock.yaml`, `bun.lockb`)
- Files outside your task's paths, or edits to `mcp/` and `workshop-data/`
- Commented-out code blocks, `console.log` debugging left behind, disabled tests or lint rules

ORBIT: rules for coding agents and humans

This file is the single source of rules for Claude Code, Codex, Cursor and people. `CLAUDE.md` and `.cursor/rules/orbit.mdc` only point here. If another document disagrees with this file, this file wins. Tell Hardik so the other document gets fixed.

0. Build model: one code writer

Only Claude, running in Ayman's session, commits code. Teammates do not commit code. Instead they:

- **decide** every customization for their area (layout, content, wording, interactions, data choices). Claude asks them in plan mode at the start of each workstream, and Ayman relays the answers.
- **review** the PRs for their area, and **test** on their own laptop.
- propose changes in the team chat or as PR comments, never as commits.

Tasks with a `builder` field in `tasks.json` are built by Claude. Their `owner` is the person who decides and reviews. Tasks without `builder` (e.g. getting the AI key, merging, rehearsal) are done by the owner themselves.

0.1 Before you change anything

1. Know **which task** you are working on (`tasks.json` , e.g. `ORB-AYM-01`). No task, no code.
2. Only edit files inside that task's `paths` .
3. Ask the task owner the customization questions before building (plan mode), and list their decisions in the PR.
4. Run `npm run check` before every commit you intend to push.

1. What ORBIT is

An AI-powered global intelligence platform on a 3D globe: **Global** → **Country** → **Event** → **Explanation** → **Ask ORBIT**. It's a 24-hour hackathon prototype, demoed live on one laptop, and it must work offline.

2. Stack (frozen, do not change or add alternatives)

Area	Choice
Frontend	Vite 8 + React 19 + TypeScript (strict)
3D globe	react-globe.gl (three.js)
Server	Express 5, run with tsx, in <code>server/</code>
Contracts	Zod 4 schemas in <code>shared/</code> , imported as <code>@shared</code>
Data fetching	TanStack Query (hooks live in each feature folder)
UI state	Zustand (<code>src/state/</code>). The selection lives in the URL (<code>src/routes.ts</code>)
Styling	CSS Modules + tokens in <code>src/styles/tokens.css</code>

Area	Choice
AI	Vercel AI SDK behind <code>server/ai/</code> , with the mock provider as default and fallback
Data	JSON seed files in <code>server/data/seed/</code> + disk cache <code>server/.cache/</code> . No database
Tests	Vitest + Testing Library
Package manager	npm only (one lockfile: <code>package-lock.json</code>). Node $\geq$ 22.12

Do not introduce Next.js, Tailwind, Redux, a database, Python services, another package manager, or another app.

3. Folder ownership (decision owners)

Each area has one **decision owner**: the person Claude asks about customization and who reviews changes there. Code in every folder is written by Claude (section 0). A task may only change files inside its own `paths`.

Owner	Owns
Hardik (integrator)	<code>shared/</code> , <code>server/app.ts</code> , <code>server/index.ts</code> , <code>server/env.ts</code> , <code>server/http.ts</code> , <code>server/testServer.ts</code> , <code>server/routes/</code> (except <code>ai.ts</code>), <code>server/data/store.ts</code> , <code>server/sources/</code> , <code>src/App.tsx</code> , <code>src/routes.ts</code> , <code>src/main.tsx</code> , <code>src/queryClient.ts</code> , <code>src/api/</code> , <code>src/test/</code> , <code>scripts/</code> , root config files, <code>package.json</code> , <code>package-lock.json</code> , <code>.env.example</code> , <code>tasks.json</code> , <code>AGENTS.md</code> , <code>CLAUDE.md</code> , <code>.cursor/</code> , <code>README.md</code> , <code>docs/</code> (except the files below)
Affan	<code>server/ai/</code> , <code>server/routes/ai.ts</code> , <code>src/features/ask/</code> , <code>src/features/insight/</code>
Arham	<code>src/styles/</code> , <code>src/ui/</code> , <code>src/globe/</code> , <code>src/layout/</code> , <code>src/state/</code> , <code>public/</code> , <code>docs/DESIGN_SYSTEM.md</code>
Ayman	<code>src/features/country/</code> , <code>src/features/event/</code> , <code>src/features/timeline/</code>
Shrey	<code>server/data/seed/</code> , <code>src/features/news/</code> , <code>src/features/health/</code> , <code>docs/DEMO.md</code>

Never touch: `mcp/`, `workshop-data/`, `.vscode/mcp.json` (hackathon organiser tooling).

Shared hot files (change them in a small, separate PR that Hardik merges first, before the feature PR that needs it): `shared/**`, `package.json`, `package-lock.json`, `src/App.tsx`, `src/routes.ts`, `src/styles/tokens.css`, `src/ui/index.ts`, `server/app.ts`, `.env.example`, `tasks.json`, `AGENTS.md`.

4. Shared contracts (`shared/`)

- Every data type is defined **once**, as a Zod schema in `shared/schemas/`: `XSchema` + `type X = z.infer<typeof XSchema>`.
- Import types and schemas only from `@shared`: `import { type Country, API_ROUTES } from '@shared'`.
- Never** write your own `interface Country`, `type Event`, etc. **Never** name a type `Event` (it clashes with the DOM). Use `OrbitEvent`.
- IDs: countries = ISO alpha-3 uppercase (`IND`). Everything else = prefix + lowercase slug: `evt_` geopolitical event, `hs_` health signal, `news_`, `src_`, `tl_`, `ins_`.
- Dates: ISO 8601 UTC strings (`2026-09-19T10:00:00Z`). Coordinates: `{ lat, lng }`. Severity: integer 1–5.
- Fields: camelCase. Enum values: lowercase strings.
- Changing a contract: follow `docs/INTEGRATION.md#contract-changes`. New fields start as `.optional()`.

5. API rules

- All routes are listed in `docs/API.md` and built with `API_ROUTES` from `@shared`. Never hand-type `/api/...` strings.
- Success responses: `{ "data": ... }`. Error responses: `{ "error": { "code", "message" } }` plus an HTTP status.
- Server: validate every input with `parseInput(schema, value)`. Throw `AppError` / `notFound()` (in `server/http.ts`). Never `res.status(...).json(...)` by hand.
- Frontend: the only `fetch` calls live in `src/api/client.ts`. Features call `apiGet` / `apiPost` inside TanStack Query hooks in their own folder.
- Query keys start with the feature name (`['country', id]`) so features never collide.
- Only the server talks to external APIs or AI providers. The browser never does.

6. Environment and secrets

- Real values go in a local `.env` (gitignored). `.env.example` lists every variable with a safe empty or default value.
- **Never** commit keys, tokens or `.env`. **Never** put a secret in a `VITE_` variable (those ship to the browser).
- Adding a variable: add it to `.env.example` **and** `server/env.ts` in the same PR (Hardik merges).

7. Dependencies

- A new package needs a reason stated in the approved plan and Hardik's OK in the chat. Claude installs it in the PR that needs it, with the lockfile change in that PR. Nobody else runs `npm install <package>`.
- Everyone only runs plain `npm install` (after pulling) to sync.
- Never delete or regenerate `package-lock.json`. Never use yarn, pnpm or bun.

8. Styling rules

- CSS Modules only: `Component.module.css` next to `Component.tsx`. No global class names except in `src/styles/global.css`.
- Use tokens: `var(--color-primary)`, `var(--space-4)` ... **No raw hex colours, px font sizes or shadows outside** tokens.css.
- Canvas/WebGL code reads tokens with `cssVar()` from `src/styles/cssVar.ts`.
- Reuse primitives from `src/ui` (Panel, Loader, ErrorState, ErrorBoundary, QueryState, ItemList, SeverityBadge). Need a new one? Ask Arham.
- Details: `docs/DESIGN_SYSTEM.md`.

9. Error handling

- Every query renders loading **and** error states. Use `<QueryState query={...} label="...">`.
- Wrap each feature section in `<ErrorBoundary title="...">` so one crash never blanks the app.
- Server: never crash on bad data from a live source or AI. Log `[sources] ...` / `[ai] ...` and fall back (cache → seed / mock).
- Logging uses a prefix: `[api]`, `[sources]`, `[ai]`, `[ui]`.

10. Testing

- `npm run check` = lint + tests + typecheck/build. It must pass before every PR.
- Each task lists the tests it needs in `tasks.json`. Put tests next to the code (`x.test.ts(x)`).
- Server/shared tests start with `// @vitest-environment node`. Frontend tests use `renderWithProviders` from `src/test/utils.tsx`.
- The globe is mocked in tests (no WebGL in jsdom). Do not remove that mock.

11. Git workflow

- `main` must always run. Only Hardik merges (Affan is backup).
- Branch per task: the `branch` field in `tasks.json` (`claude/<topic>` for Claude-built tasks, e.g. `claude/globe`).
- Commit messages: `<area>: <what changed>` (e.g. `globe: add severity rings`). Small commits.
- Before opening a PR: `git pull origin main`, fix conflicts in YOUR files only, run `npm run check`.
- PR title: `[ORB-XXX-NN] short summary`. Squash merge.
- Full rules: `docs/INTEGRATION.md`.

12. What agents must NOT do

- Invent tasks, features, routes, fields or packages that are not in `tasks.json` / `docs/API.md`.
- Make a customization decision (layout, wording, content, interaction, data choice) without asking the task owner.
- Edit files outside the current task's owner folders "while they're at it".
- Change the stack, add a framework, or restructure folders.
- Disable lint rules, delete tests or loosen TypeScript to make `npm run check` pass.
- Commit `.env`, `keys`, `dist/`, `server/.cache/` or `node_modules/`.

ORBIT design system

Owner: Arham. The tokens live in `src/styles/tokens.css` (the only place raw values may appear). The primitives live in `src/ui/`.

Direction (updated 2026-09-19): Apple-style shapes and layout. Frosted translucent materials, hairline borders, large rounded corners, capsule controls with tinted fills, San Francisco typography, soft depth instead of neon glow. Colours stay ORBIT's: orange highlights, cyan accents, in Apple system tints. A floating toolbar capsule sits at the top, and headlines run in a floating "Live" pill at the bottom. Effects stay sleek, not busy. Futuristic, cinematic, clean, like an intelligence centre. The globe is the hero, and panels float over it as dark glass.

Tokens

Colour

Token	Use
<code>--color-bg</code> , <code>--color-bg-elevated</code>	Page background, raised areas
<code>--color-surface</code> , <code>--color-surface-hover</code>	Glass panels (semi-transparent over the globe)
<code>--color-border</code> , <code>--color-border-strong</code>	Hairlines, focused/active outlines
<code>--color-text</code> , <code>--color-text-muted</code> , <code>--color-text-faint</code>	Primary, secondary, tertiary text
<code>--color-primary</code> , <code>--color-primary-dim</code>	Electric cyan: links, focus, geopolitical
<code>--color-accent</code>	Amber: Ask ORBIT, health, highlights
<code>--color-alert</code>	Orange: critical, errors
<code>--color-highlight</code>	Orange: panel titles, selected items
<code>--color-success</code> , <code>--color-danger</code>	Status only
<code>--severity-1</code> ... <code>--severity-5</code>	The shared 1–5 severity scale (cyan → amber → orange)
<code>--kind-geopolitical</code> , <code>--kind-health</code>	Event kind colours
<code>--globe-*</code>	Orange globe: ocean, land, hover, selected, borders, atmosphere, ring

Typography

Token	Value
<code>--font-display</code>	SF Pro Display (Apple) → Inter → Segoe UI
<code>--font-body</code> , <code>--font-label</code>	SF Pro Text (Apple) → Inter → Segoe UI
<code>--font-mono</code>	SF Mono → JetBrains Mono (code only)

Token	Value
<code>--text-xs ... --text-3xl</code>	0.75rem → 2.75rem scale
<code>--weight-regular/medium/bold</code>	400 / 500 / 700
<code>--tracking-label</code>	Letter spacing for uppercase eyebrow labels
<code>--leading-tight</code> , <code>--leading-body</code>	Line heights

Apple devices use San Francisco. Elsewhere Inter loads from Google Fonts, and offline it falls back to Segoe UI.

Spacing, borders, radii

- Spacing on a 4px grid: `--space-1` (4px) ... `--space-8` (64px).
- `--border-width` (1px). Radii: `--radius-sm` 8px, `--radius-md` 14px (tiles, rows), `--radius-lg` 22px (panels), `--radius-xl` 28px, `--radius-pill`.

Glass panels

`--glass-bg` (gradient fill), `--glass-border`, `--glass-highlight` (inner top highlight), `--glass-accent-border` (amber variant). Panels are dark glass: translucent, blurred, thin cyan hairline, 14px radius.

Shadows and glows

`--shadow-panel` (panel depth), `--glow-primary` (cyan), `--glow-accent` (amber), `--glow-alert` (orange), `--glow-soft` (panel hover), `--blur-panel` (glass blur). Use glows sparingly: for focus, hover and severity 5 only.

Motion and layers

- `--duration-fast/base/slow` (150/250/600ms) with `--ease-out`. `--duration-layout` (700ms) with `--ease-inout` for the globe squeeze between the global and detail views. They all become 0 when the user prefers reduced motion.
- `--z-globe` < `--z-overlay` < `--z-panel` < `--z-modal`.
- Layout sizes: `--panel-width` (420px, right column), `--center-width` (760px, detail view), `--mini-globe-size` (220px), `--drawer-width` (420px, Ask ORBIT drawer), `--topbar-height` (56px).

Globe

Realistic Earth (decided 2026-09-19): NASA Blue Marble day texture, terrain bump map, shiny oceans (specular water map) and a starfield page background. Textures are public domain (via the globe.gl examples) and live in `public/textures/` so the demo works offline. Countries are faint white hairlines. Hover lightens a country, and the selected country is filled orange (`--globe-land-selected`), which keeps orange as the highlight colour.

- **Pins:** every event gets a thin pin coloured by severity (`--severity-N`). More severe events get taller pins, but pins always stay short.
- **Rings:** only severity 4–5 events get a small ripple ring that fades as it grows (`src/globe/globeStyle.ts`, `-globe-ring`).
- **Motion:** the global view rotates slowly when idle. Rotation pauses while the user drags and resumes after 4 s. In detail views the globe becomes the bottom-left mini globe. In the global view it is centred left of the panel column.

- **Ripple arcs:** animated dashed arcs from an event to each country it affects, coloured by channel (`--channel-energy` orange, `--channel-finance` cyan, `--channel-health` red, `--channel-trade` green, `--channel-shipping` blue, `--channel-security` yellow). In detail views only the arcs touching the selected country remain.
- **Tooltips:** `.globe-tooltip` in `global.css` (the globe library renders labels as HTML strings, and the text is escaped).

Primitives (`src/ui` , `import from ../../ui`)

Component	Use
<code>Panel</code>	Every overlay section (glass). Props: <code>eyebrow</code> , <code>title</code> , <code>actions</code> , <code>tone</code> (<code>default</code> or <code>accent</code> for AI content), <code>children</code>
<code>QueryState</code>	Renders loading / error / data for a TanStack query: <code><QueryState query={q} label="country">{(data) => ...}</QueryState></code>
<code>Loader</code>	Loading state (<code>role="status"</code>)
<code>ErrorState</code>	Error state with optional retry (<code>role="alert"</code>)
<code>ErrorBoundary</code>	Wrap each feature section so a crash stays contained
<code>ItemList</code>	Vertical list with dividers and a required empty state
<code>SeverityBadge</code>	Shows severity 1–5 with its colour and label (severity 5 pulses slowly)
<code>Button</code>	Variants <code>primary</code> (cyan), <code>accent</code> (amber: Ask ORBIT, AI), <code>ghost</code> . Sizes <code>sm</code> , <code>md</code> . <code>iconOnly</code> requires <code>aria-label</code>
<code>Tag</code>	Small uppercase pill. <code>kind</code> (<code>geopolitical</code> / <code>health</code>) colours it; without <code>kind</code> it is neutral
<code>StatTile</code>	One labelled figure (<code>label</code> , <code>value</code> , optional <code>hint</code>). Put several in a grid for a stat strip

New primitives are decided with Arham (the design decision owner).

Common UI states (every data view needs all four)

State	How
Loading	<code>QueryState</code> / <code>Loader</code>
Error	<code>QueryState</code> / <code>ErrorState</code> (with retry)
Empty	<code>ItemList</code> <code>empty</code> text, or a sentence in muted text
Success	The content

Conventions

- One `Component.module.css` next to each component. Class names are camelCase (`styles.askButton`).
- Only tokens: `color: var(--color-text-muted)` , never `#8aa0b8` . The same goes for spacing, radii and shadows.
- Canvas/WebGL (globe): read tokens with `cssVar('--globe-land')` and `severityColor(severity)` from `src/styles/cssVar.ts` .

- Animations: define `@keyframes` inside the component's own `.module.css` (CSS Modules rename keyframes, so global ones can't be referenced).
- Text: sentence case. Eyebrow labels are uppercase via CSS, not typed in capitals.
- Accessibility: visible focus (global `:focus-visible` ring), `aria-label` on icon-only buttons, and colour never the only signal (severity badges also show the label and number).
- Numbers: `toLocaleString()` for large values. Dates: show them at their `datePrecision`.

ORBIT personal LLM allocator

Paste the prompt below into your own LLM (Claude Code, Codex, Cursor, ChatGPT...). Replace `<YOUR NAME>` with one of: **Arham, Ayman, Affan, Shrey, Hardik**. If your LLM cannot read files from the repository, paste the contents of `tasks.json`, `AGENTS.md`, `docs/PLAYBOOK.md`, `docs/API.md` and `docs/INTEGRATION.md` after the prompt.

Build model reminder: Claude, running in Ayman's session, writes all the code. For tasks with a `builder` field, you are the **decision owner**: you answer Claude's customization questions and review/test the result. You don't write that code yourself. Tasks without `builder` are yours to do.

You are my personal task allocator for the ORBIT hackathon project. My name is: <YOUR NAME>

SOURCES (the only truth, read all of them before answering):

1. tasks.json - the authoritative task list (read "\$comment", "team" and every task)
2. AGENTS.md - rules, especially section 0 (build model) and section 3 (decision owners)
3. docs/PLAYBOOK.md - overview, roles, dependency graph, definition of done
4. docs/INTEGRATION.md - PRs, reviews, contract changes
5. docs/API.md and docs/DESIGN_SYSTEM.md - only when my decisions concern data or UI

HARD RULES:

- Use ONLY tasks from tasks.json whose "owner" is exactly my name. Never invent, merge, split or rename tasks.
- A task WITH a "builder" field is built by Claude in Ayman's session. My job there is to DECIDE (answer customization questions: layout, content, wording, interactions, data choices) and REVIEW/TEST. Never tell me to write or commit code for it.
- A task WITHOUT "builder" is mine to do myself.
- If something I ask for is not in tasks.json, say "Not in tasks.json - ask Hardik" instead of making it up.
- If my name is not one of Arham, Ayman, Affan, Shrey, Hardik, stop and ask me for the correct name.
- Quote IDs, branches and acceptance criteria exactly as written in the sources.

OUTPUT (in this order, concise, using headings):

1. My role: my "team" entry in tasks.json and my row in the roles table in docs/PLAYBOOK.md section 9.
2. My tasks: a table sorted by priority (P0, P1, P2), then targetWindow:
ID | title | priority | targetWindow | status | who builds (me / Claude) | dependsOn (and whether each is done).
3. Do now: the tasks I do myself (no "builder") that are not done and whose dependencies are done, with description, acceptance criteria (verbatim) and handoff.
4. Decide soon: for each of my Claude-built tasks that is not done, in order:
 - What it builds (description, one paragraph)
 - The customization decisions I should prepare BEFORE Claude asks (derive them from the description and acceptance criteria: e.g. what to show, in what order, wording, visual style, data choices). List them as questions I can think about now. Do not answer them for me.
 - What "done" means (acceptance criteria, verbatim)
5. How to review: when the PR for one of my tasks appears, (a) read the PR description and check my decisions are listed, (b) pull the branch and run `git fetch && git checkout <branch> && npm install && npm run dev`, (c) click through my area and check each acceptance criterion, (d) post feedback as PR comments, (e) tell Hardik "approved" in the chat. Never push commits.
6. Rules I must remember: the 5 most relevant rules from AGENTS.md for me (always include: no secrets in code or chat, and never commit code).

Then ask me which customization decision I want help thinking through.

How the allocator stays correct

- It only reads the files above, so updating tasks.json (through a PR merged by Hardik) updates every person's allocation.
- npm test validates tasks.json (unique IDs, valid owners, existing dependencies, no cycles, everyone has a task).
- When Hardik merges a PR, the task status is set to done in tasks.json, so the allocator moves on. Pull main to see it.